Applied Reinforcement Learning

Dynamic Programming Exercises

Riccardo Berta

2026.10.29

0.1 Exercises on Dynamic Programming

These exercises follow the progression of the lecture notes: evaluate a fixed policy, improve it using one-step lookahead, and compute an optimal value function. They then examine why different policies can be equally good, how policy iteration avoids exhaustive search, and how discounting can change the preferred action.

All transition probabilities and rewards are known: the calculations use the MDP model rather than sampled experience. Lowercase v and q denote value functions, while uppercase V denotes an approximation computed by an iterative algorithm. Terminal states have value zero because no further rewards are received after termination; the reward on the transition into a terminal state is still included. Unless stated otherwise, assume a finite discounted MDP with bounded rewards.

0.1.1 Exercise 3.1 — Exact policy evaluation

Consider an MDP with two nonterminal states, S1 and S2, and one terminal state. A fixed deterministic policy selects the following transitions:

- From S1, the agent moves to S2 with probability 1 and receives reward -1.
- From S2, the agent terminates with probability 0.5 and receives reward $+10$; otherwise, it returns to S1 and receives reward -1.

There is no discounting:

$\gamma = 1$

Write the Bellman expectation equation for each nonterminal state and solve for the two state values. Explain why these values remain finite even without discounting.

Step 1 — Write the Bellman expectation equations

For a deterministic policy, there is no need to average over actions: in each state, the policy selects a single action. We average only over its possible outcomes:

$$v_\pi(s) = \sum_{s',r} p(s', r \mid s, \pi(s)) \left[r + \gamma v_\pi(s') \right]$$

From S1, the agent pays a cost of 1 and then continues from S2:

$$v_\pi(S_1) = -1 + v_\pi(S_2)$$

From S2, the successful transition contributes its immediate reward of 10 and no continuation value. The other transition contributes -1 plus the value of returning to S1:

$$v_\pi(S_2) = 0.5(10 + 0) + 0.5\left[-1 + v_\pi(S_1)\right] = 4.5 + 0.5 v_\pi(S_1)$$

Step 2 — Solve the linear system

Substitute the equation for S1 into the equation for S2:

$v_\pi(S_2) = 4.5 + 0.5\,[-1 + v_\pi(S_2)] = 4 + 0.5 v_\pi(S_2)$

Collecting terms gives:

$0.5 v_\pi(S_2) = 4 \quad \Longrightarrow \quad v_\pi(S_2) = 8$

Back-substitution gives the remaining value:

$v_\pi(S_1) = -1 + 8 = 7$

The value of S1 is exactly one unit lower because starting there adds a transition with reward -1 before reaching S2.

Key concept

For a fixed policy, the Bellman expectation equations form a linear system: the unknown state values appear only linearly. Solving this system computes the expected return over all possible future trajectories without enumerating them.

0.1.2 Exercise 3.2 — Iterative policy evaluation

Evaluate the policy from Exercise 3.1 iteratively, starting from zero:

$V_0(S_1) = 0$
$V_0(S_2) = 0$

Use synchronous updates: compute every value in a sweep from the values at the end of the previous sweep, as in the notes:

$V_{k+1}(s) = \sum_{s',r} p(s', r \mid s, \pi(s))\,[r + \gamma V_k(s')]$

Perform four complete sweeps. Compare the estimates with the exact solution.

Step 1 — Specialize the update equations

Replace the unknown true successor values in the Bellman equations with their current estimates:

$V_{k+1}(S_1) = -1 + V_k(S_2)$

$V_{k+1}(S_2) = 4.5 + 0.5\,V_k(S_1)$

Both right-hand sides use sweep k. Using a newly computed value from sweep k+1 would instead give an in-place update and a different sequence of intermediate estimates.

Step 2 — Perform four sweeps

Sweep	Estimate for S1	Estimate for S2
0	0	0
1	$-1 + 0 = -1$	$4.5 + 0.5 \times 0 = 4.5$
2	$-1 + 4.5 = 3.5$	$4.5 + 0.5 \times (-1) = 4$
3	$-1 + 4 = 3$	$4.5 + 0.5 \times 3.5 = 6.25$
4	$-1 + 6.25 = 5.25$	$4.5 + 0.5 \times 3 = 6$

The estimates have not yet reached the exact values of 7 and 8. They also do not increase at every sweep: starting from zero does not imply monotonic improvement when some rewards are negative.

Key concept

Iterative policy evaluation is an example of bootstrapping: each estimate is updated using estimates of successor states. It approaches the same solution as the linear system without requiring a direct linear solve. The index k counts computational sweeps, not environment time steps or episodes. Each sweep uses the known transition model; the agent does not need to interact with the environment. For discounted finite MDPs, the contraction proof in the notes guarantees convergence. This undiscounted example converges because its terminating dynamics still make errors vanish.

0.1.3 Exercise 3.3 — Policy improvement

In state X, the agent can choose between two actions. Both give immediate reward zero, and the discount factor is 0.9:

- Action a1 moves deterministically to Y, whose value under the current policy is 5.
- Action a2 moves deterministically to Z, whose value under the current policy is 8.

The current policy selects a1 in X. Construct a new policy that selects the greedy action in X and leaves the policy unchanged everywhere else.

1. Compute both action values under the current policy.
2. Choose the greedy action.
3. Verify the policy-improvement condition in every state and explain what it guarantees about the new policy.

Step 1 — Compute the action values

An action value evaluates taking that action once and then following the current policy. The one-step lookahead is:

$$q_\pi(X,a) = \sum_{s',r} p(s',r \mid X,a)\,[r + \gamma v_\pi(s')]$$

The transitions are deterministic and the immediate rewards are zero:

$$q_\pi(X,a_1) = 0 + 0.9 \cdot 5 = 4.5$$

$$q_\pi(X,a_2) = 0 + 0.9 \cdot 8 = 7.2$$

Since the current policy selects a1, its value in X is:

$$v_\pi(X) = q_\pi(X,a_1) = 4.5$$

Step 2 — Choose the greedy action

Policy improvement selects an action with the largest action value under the current policy:

$$\pi'(X) \in \arg\max_a q_\pi(X,a)$$

Here, the maximizer is unique:

$$7.2 > 4.5 \quad \Longrightarrow \quad \pi'(X) = a_2$$

In every other state, the new policy keeps the current action.

Step 3 — Apply the policy-improvement theorem

The theorem requires the new action to be at least as good as continuing with the old policy in every state:

$$q_\pi(s,\pi'(s)) \geq v_\pi(s) \quad \forall s \quad \Longrightarrow \quad v_{\pi'}(s) \geq v_\pi(s) \quad \forall s$$

In X, the inequality is strict:

$$q_\pi(X,\pi'(X)) = 7.2 > 4.5 = v_\pi(X)$$

In all other states, the policies choose the same action, so equality holds:

$$q_\pi(s,\pi'(s)) = q_\pi(s,\pi(s)) = v_\pi(s), \qquad s \neq X$$

The new policy is therefore no worse from any state and strictly better from X.

Key concept

Policy improvement turns the values of the current policy into a new policy by acting greedily with respect to one-step lookahead. The values used do not need to be optimal: they only need to describe the policy being improved. The new policy is never worse, and it is strictly better unless the current policy is already optimal. Alternating evaluation and improvement is the core of policy iteration.

0.1.4 Exercise 3.4 — Policy iteration

Consider an MDP with two nonterminal states, A and B, and a terminal state G. All transitions are deterministic, and the discount factor is 0.9:

State	Action	Next state	Reward
A	stop	G	+1
A	go	B	0
B	wait	B	0
B	finish	G	+2

Start from the following policy:

$\pi_0(A) = \text{stop}, \qquad \pi_0(B) = \text{wait}$

Apply policy iteration: evaluate the current policy exactly, then improve it simultaneously in every state using one-step lookahead. Keep the terminal value at zero. If actions tie, keep the current action.

1. Compute the initial policy's state values and perform the first improvement step.
2. Evaluate the improved policy and perform another improvement step. Explain why the action in A does not change during the first improvement but does change during the second.
3. Continue until the policy is stable. Give the final policy and value function, and explain why stability establishes optimality.
4. Count the policy changes and the complete evaluation–improvement cycles, including the final stability check.

Step 1 — Evaluate and improve the initial policy

For a deterministic policy, policy evaluation solves the Bellman expectation equations:

$$v_{\pi_k}(s) = \sum_{s',r} p(s', r \mid s, \pi_k(s)) \left[r + \gamma v_{\pi_k}(s') \right]$$

Here, k indexes successive policies. Evaluation computes the value of the current policy before any actions are changed. The initial policy stops immediately in A, receiving reward 1. In B, it waits forever and receives only zero rewards. Substituting its actions into the general equation gives:

$v_{\pi_0}(A) = 1$

$$v_{\pi_0}(B) = 0 + 0.9\, v_{\pi_0}(B) \quad \Longrightarrow \quad v_{\pi_0}(B) = 0$$

To improve the policy, compute the value of taking each available action once and then following the current policy:

$$q_{\pi_k}(s,a) = \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_{\pi_k}(s') \right]$$

For the initial policy, these one-step lookahead calculations give:

$q_{\pi_0}(A, \text{stop}) = 1$
$q_{\pi_0}(A, \text{go}) = 0 + 0.9 \cdot 0 = 0$
$q_{\pi_0}(B, \text{wait}) = 0 + 0.9 \cdot 0 = 0$
$q_{\pi_0}(B, \text{finish}) = 2$

The improvement step selects a maximizing action in each state:

$$\pi_{k+1}(s) \in \arg\max_a q_{\pi_k}(s,a)$$

Applying this rule to the initial policy changes only the action in B:

$\pi_1(A) = \text{stop}$
$\pi_1(B) = \text{finish}$

Step 2 — Evaluate the new policy and improve again

Under the new policy, the agent stops in A and finishes immediately in B:

$v_{\pi_1}(A) = 1$
$v_{\pi_1}(B) = 2$

Repeat the one-step lookahead with these updated values:

$q_{\pi_1}(A, \text{stop}) = 1$
$q_{\pi_1}(A, \text{go}) = 0 + 0.9 \cdot 2 = 1.8$
$q_{\pi_1}(B, \text{wait}) = 0 + 0.9 \cdot 2 = 1.8$
$q_{\pi_1}(B, \text{finish}) = 2$

The second improvement changes the action in A while keeping finish in B:

$\pi_2(A) = \text{go}$
$\pi_2(B) = \text{finish}$

Step 3 — Check stability and count the cycles

Evaluate the latest policy:

$v_{\pi_2}(A) = 0 + 0.9 \cdot 2 = 1.8$
$v_{\pi_2}(B) = 2$

Another improvement step compares stop with go in A, and wait with finish in B. The action values remain 1 versus 1.8 in A and 1.8 versus 2 in B. Neither selected action changes, so the policy is stable. In general, policy iteration stops when an improvement step leaves the policy unchanged in every nonterminal state:

$\pi_{k+1}(s) = \pi_k(s) \quad \forall s$

At stability, the current policy already selects a maximizing action with respect to its own value function. Combining evaluation and greedy improvement therefore gives the Bellman optimality equation. For the policy obtained here:

$v_{\pi_2}(s) = \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_{\pi_2}(s') \right], \qquad s \in \{A, B\}$

Since the Bellman optimality equation has a unique solution for this finite discounted MDP, the stable policy is optimal:

$\pi_*(A) = \text{go}$
$\pi_*(B) = \text{finish}$

$v_*(A) = 1.8$
$v_*(B) = 2$

The algorithm performed two policy changes and three complete evaluation–improvement cycles:

Cycle	Policy evaluated: action in A, action in B	Value in A	Value in B	Result of improvement
1	stop, wait	1	0	Change B to finish
2	stop, finish	1	2	Change A to go
3	go, finish	1.8	2	No change: stop the algorithm

The final cycle confirms stability; it does not produce another policy change.

Key concept

Policy iteration repeatedly alternates evaluation and improvement. Evaluation computes the consequences of the current policy, and improvement uses those values to choose better actions. Improving one state's behavior can make a different action attractive in another state during a later cycle. The state values never decrease along this sequence: A's value progresses from 1 to

1 to 1.8, while B's progresses from 0 to 2 to 2. An unchanged action in one state does not mean that action is already optimal. Optimality is established when a complete improvement step leaves the policy unchanged in every state.

0.1.5 Exercise 3.5 — Value iteration

Consider two nonterminal states, A and B, and a terminal goal G. The discount factor is 0.9, and the transitions are deterministic:

- In A, ”right” moves to B and ”stay” remains in A, both give reward zero.
- In B, ”right” moves to G with reward +1, while ”left” moves to A with reward zero.

Initialize all values to zero and keep the terminal value at zero. Perform three synchronous value-iteration sweeps, then extract a greedy policy.

Step 1 — Apply the Bellman optimality update

Value iteration evaluates every available action using the current successor estimates and keeps the largest result:

$$V_{k+1}(s) = \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma V_k(s') \right]$$

For this MDP, the two updates simplify to:

$$V_{k+1}(A) = \max \{0.9 V_k(B),\ 0.9 V_k(A)\}$$

$$V_{k+1}(B) = \max \{1,\ 0.9 V_k(A)\}$$

The first and second entries correspond to right and stay in A, and right and left in B. The terminal reward in B is 1, with no continuation value after reaching G.

Sweep	Estimate for A	Estimate for B
0	0	0
1	$\max\{0, 0\} = 0$	$\max\{1, 0\} = 1$
2	$\max\{0.9, 0\} = 0.9$	$\max\{1, 0\} = 1$
3	$\max\{0.9, 0.81\} = 0.9$	$\max\{1, 0.81\} = 1$

The first sweep incorporates the goal reward into B. The second propagates its discounted value to A. The third confirms that the values form a fixed point of the Bellman optimality update:

$v_*(A) = 0.9$
$v_*(B) = 1$

Step 2 — Extract a greedy policy

Once the optimal values are known, choose an action attaining the largest one-step lookahead in each nonterminal state:

$$\pi_*(s) \in \arg\max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_*(s') \right]$$

In A, compare the two actions:

$q_*(A, \text{right}) = 0 + 0.9 \cdot 1 = 0.9$
$q_*(A, \text{stay}) = 0 + 0.9 \cdot 0.9 = 0.81$

In B, the comparison is:

$q_*(B, \text{right}) = 1 + 0.9 \cdot 0 = 1$
$q_*(B, \text{left}) = 0 + 0.9 \cdot 0.9 = 0.81$

Right is the unique maximizing action in both states:

$\pi_*(A) = \text{right}$
$\pi_*(B) = \text{right}$

Starting from A, the resulting trajectory is:

$A \longrightarrow B \longrightarrow G$

Key concept

Value iteration applies the Bellman optimality operator directly. Unlike policy iteration, it does not evaluate an intermediate policy to convergence before considering better actions: every sweep already includes a maximization over actions. An explicit policy can be extracted at the end. This small example reaches the exact fixed point after finitely many sweeps. In general, discounted value iteration converges only in the limit, and the implementation stops when the largest change in a sweep falls below a chosen tolerance.

0.1.6 Exercise 3.6 — How discounting changes the optimal action

From a start state s, the agent chooses between two actions. After this choice, there are no further decisions:

- "SOONER" gives reward +1 on the next transition and terminates the episode.
- "LATER" gives reward zero on the next transition, followed by reward +2 on the transition after that, and then terminates the episode.

Find the discount factor at which the agent is indifferent between the actions. Determine which action is optimal below, above, and at this threshold.

Step 1 — Compute the action values

The return discounts rewards according to how many transitions separate them from the next reward:

$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$

The next reward is not discounted. Since there are no further decisions, the optimal action values are simply:

$q_*(s, \text{SOONER}) = 1$

$q_*(s, \text{LATER}) = 0 + \gamma \cdot 2 = 2\gamma$

These are action values because they compare two choices from the same state; they are not values of two different states.

Step 2 — Find the indifference point

Set the expected returns equal:

$q_*(s, \text{SOONER}) = q_*(s, \text{LATER}) \quad \Longleftrightarrow \quad 1 = 2\gamma$

The threshold discount factor is:

$\gamma = \frac{1}{2}$

Step 3 — Determine the optimal action

Below the threshold, the delayed reward is discounted enough to make SOONER preferable:

$0 \leq \gamma < \frac{1}{2} \quad \Longrightarrow \quad q_*(s, \text{SOONER}) > q_*(s, \text{LATER})$

Above the threshold, the larger delayed reward makes LATER preferable:

$\frac{1}{2} < \gamma \leq 1 \quad \Longrightarrow \quad q_*(s, \text{LATER}) > q_*(s, \text{SOONER})$

At the threshold, both actions are optimal:

$\gamma = \frac{1}{2} \quad \Longrightarrow \quad q_*(s, \text{SOONER}) = q_*(s, \text{LATER}) = 1$

The endpoint with no discounting is well defined here because either choice terminates within two transitions.

Key concept

The discount factor is part of the objective, not merely an algorithm setting. Changing it can change the optimal policy even when the transitions and rewards stay the same. A smaller

discount factor gives less weight to delayed rewards and can make a smaller immediate reward preferable to a larger reward received later.

0.1.7 Exercise 3.7 — Policy iteration versus brute-force search

Consider an MDP with 10 nonterminal states and four actions available in each state. Count stationary deterministic policies: each policy assigns one fixed action to each state.

An exhaustive brute-force search evaluates every such policy. Suppose policy iteration instead makes five policy changes before reaching an optimal policy, which it then evaluates and checks for stability.

1. How many distinct policies does each method evaluate?
2. What is the ratio between these counts?
3. Explain why this ratio is not automatically the ratio between execution times.

Step 1 — Count the policies evaluated

There are four independent action choices for each of the 10 states:

$N_{\text{BF}} = |\mathcal{A}|^{|\mathcal{S}|} = 4^{10} = 1{,}048{,}576$

Policy iteration evaluates the initial policy and the five successive policies produced by improvement:

$N_{\text{PI}} = K + 1 = 5 + 1 = 6$

The final policy is evaluated before a further improvement check confirms stability. This check makes no additional policy change. Thus, five policy changes correspond to six evaluation–improvement cycles in the implementation used in the notes.

Step 2 — Interpret the ratio

Divide the number of policies examined by exhaustive search by the number evaluated along the policy-iteration sequence:

$$\frac{N_{\text{BF}}}{N_{\text{PI}}} = \frac{1{,}048{,}576}{6} \approx 174{,}763$$

In this example, exhaustive search evaluates roughly 175,000 times as many policies. This measures a reduction in the number of policies evaluated, not a guaranteed speedup by the same factor. Different policies can require different numbers of evaluation sweeps, and policy iteration also performs improvement steps.

Key concept

Policy iteration uses the value of the current policy to construct the next one, producing a directed sequence of improving policies. It avoids enumerating the entire policy space, whose size grows exponentially with the number of states. The assumed five policy changes describe this particular example, they are not a general bound on the number of changes required. To compare computational effort, we must also account for the work within each evaluation and improvement step.